

| Algorithmics | Student information     | Date                                                                                                                 | Number of session |
|--------------|-------------------------|----------------------------------------------------------------------------------------------------------------------|-------------------|
|              | UO: 307423              | 18/02                                                                                                                | 0                 |
|              | Surname: Miranda Martín |  Escuela de Ingeniería Informática |                   |
|              | Name: Mariana Patricia  |                                                                                                                      |                   |



## Activity 1. [Factor 1: Problem Size]

Make a table reflecting the execution times of the PythonA1.py module for the exposed values of n (5000, 10000, 20000, 40000, 80000, 160000, 320000, 640000). If for any n it takes more than 60 seconds you can indicate OoT ("Out of Time"), both in this section and in subsequent sections.

| Lab Computer PythonA1 | n         | 5000 | 10000 | 20000 | 40000 | 80000 | 160000 | 320000 | 640000 |
|-----------------------|-----------|------|-------|-------|-------|-------|--------|--------|--------|
|                       | time (ms) | 537  | 2150  | 8918  | 34245 |       |        |        |        |

## Activity 2. [Factor 2: Computer Performance]

Make a table that reflects, at least for two computers to which you have access, the execution times of the PythonA1.py module for the exposed values of n (5000, 10000, 20000, ..., 640000). Clearly indicate which CPU and RAM memory you are using in each test.

Lab Computer CPU and RAM – Intel(R) Core(TM) i5-10500 CPU @ 3.10GHz, 16 GB

Laptop CPU and RAM – AMD Ryzen 7 5800H with Radeon Graphics, 16 GB

| Lab Computer PythonA1 | n         | 5000 | 10000 | 20000 | 40000 | 80000 | 160000 | 320000 | 640000 |
|-----------------------|-----------|------|-------|-------|-------|-------|--------|--------|--------|
|                       | time (ms) | 537  | 2150  | 8918  | 34245 |       |        |        |        |
| Laptop Python A1      | n         | 5000 | 10000 | 20000 | 40000 | 80000 | 160000 | 320000 | 640000 |
|                       | time (ms) | 3791 | 15679 |       |       |       |        |        |        |

| Algorithmics | Student information     | Date  | Number of session |
|--------------|-------------------------|-------|-------------------|
|              | UO: 307423              | 18/02 | 0                 |
|              | Surname: Miranda Martín |       |                   |
|              | Name: Mariana Patricia  |       |                   |

## Activity 3. [Factor 3: Implementation Environment]

Program a class named JavaA1.java, which uses the same A1 algorithm to find out if a number is a prime number, as in PythonA1.py. Then, a table must be made reflecting the execution times of JavaA1.java for the same values of n (5000, 10000, 20000, ..., 640000) for both cases in two different columns (WITHOUT AND WITH OPTIMIZATION). Finally, compare these times with those obtained in Python (in a previous section) for that same algorithm A1.

|                             |           |      |       |       |       |       |        |        |        |
|-----------------------------|-----------|------|-------|-------|-------|-------|--------|--------|--------|
| PythonA1                    | n         | 5000 | 10000 | 20000 | 40000 | 80000 | 160000 | 320000 | 640000 |
|                             | time (ms) | 537  | 2150  | 8918  | 34245 |       |        |        |        |
| JavaA1 with optimization    | n         | 5000 | 10000 | 20000 | 40000 | 80000 | 160000 | 320000 | 640000 |
|                             | time (ms) | 29   | 96    | 388   | 1419  | 5828  | 23018  |        |        |
| JavaA1 without optimization | n         | 5000 | 10000 | 20000 | 40000 | 80000 | 160000 | 320000 | 640000 |
|                             | time (ms) | 96   | 359   | 1394  | 5434  | 21473 |        |        |        |

The same program executed in Java takes significantly less time than in Python. Moreover, if we execute the algorithm in Java with optimization (default setting), you can see that it's faster than if we were to execute the program without optimization (-Xint).

| Algorithmics | Student information     | Date  | Number of session |
|--------------|-------------------------|-------|-------------------|
|              | UO: 307423              | 18/02 | 0                 |
|              | Surname: Miranda Martín |       |                   |
|              | Name: Mariana Patricia  |       |                   |

## Activity 4. [Factor 4: Algorithm That Is Used]

Make a table reflecting the execution times of the modules PythonA2.py and PythonA3.py, for the same values of n (5000, 10000, 20000, ..., 640000).

Codify those same algorithms A2 and A3 in Java, in two classes named respectively JavaA2.java and JavaA3.java. Make a table that reflects the execution times of the classes JavaA2.java and JavaA3.java, for the values of n (5000, 10000, 20000, ..., 640000) WITHOUT OPTIMIZATION of the Java program and WITH OPTIMIZATION of the Java program.

|                             |           |      |       |       |       |       |        |        |        |
|-----------------------------|-----------|------|-------|-------|-------|-------|--------|--------|--------|
| PythonA2                    | n         | 5000 | 10000 | 20000 | 40000 | 80000 | 160000 | 320000 | 640000 |
|                             | time (ms) | 67   | 260   | 960   | 3578  | 13300 | 50922  |        |        |
| JavaA2 with optimization    | n         | 5000 | 10000 | 20000 | 40000 | 80000 | 160000 | 320000 | 640000 |
|                             | time (ms) | 7    | 14    | 43    | 152   | 560   | 2052   | 7807   | 29500  |
| JavaA2 without optimization | n         | 5000 | 10000 | 20000 | 40000 | 80000 | 160000 | 320000 | 640000 |
|                             | time (ms) | 12   | 41    | 147   | 543   | 2015  | 7547   | 28456  |        |
| PythonA3                    | n         | 5000 | 10000 | 20000 | 40000 | 80000 | 160000 | 320000 | 640000 |
|                             | time (ms) | 32   | 127   | 468   | 1771  | 6756  | 25109  |        |        |
| JavaA3 with optimization    | n         | 5000 | 10000 | 20000 | 40000 | 80000 | 160000 | 320000 | 640000 |
|                             | time (ms) | 13   | 33    | 136   | 545   | 2156  | 8177   | 32435  |        |
| JavaA3 without optimization | n         | 5000 | 10000 | 20000 | 40000 | 80000 | 160000 | 320000 | 640000 |
|                             | time (ms) | 41   | 156   | 619   | 2796  | 9999  | 40822  |        |        |

Finally, draw final conclusions by comparing the times previously obtained: with Python, with Java WITHOUT OPTIMIZATION and with Java WITH OPTIMIZATION.

As we saw in Activity 3, the algorithms executed in Python are drastically the slowest ones. Then, if we compare both Java execution modes, i.e., with optimization and without optimization, the first one takes less time.